

Fix di un lockdep warning in XFS — inconsistent softirq lock state in `igrab()`

Francesco Caligiuri — Matricola 207688 — Kernel Hacking, A.A. 2025/2026 — Prof. Paolo Valente

1. Bug

Bug syzkaller 5eb0d61dfb76ca12670c — **WARNING: inconsistent lock state** su `inode->i_lock` in XFS. `lockdep` segnala un possibile deadlock IRQ-inversion: la `mount` di XFS (`xfs_mountfs` → `xfs_iget` → `unlock_new_inode`) acquisisce `i_lock` in process context con softirq abilitati (SOFTIRQ-ON-W); il completamento di una read bio sul loop device, in softirq (`blk_done_softirq` → `iomap_read_end_io` → `iomap_finish_folio_read` → `fserror_report` → `igrab`), tenta di acquisire la stessa classe di lock (SOFTIRQ-IN-W). Se un softirq colpisce la stessa CPU mentre `i_lock` è tenuto dal mount, si ha deadlock.

2. Riproduzione

Ambiente: host macOS (Apple Silicon), VM Debian ARM64 (UTM), dentro la VM si esegue QEMU x86_64 con i binari pre-compilati che syzbot fornisce per il commit segnalato.

```
# download degli asset dal bug report syzbot
# bzImage-dc855b77          - kernel pre-compilato syzkaller #0
# non_bootable_disk-dc855b77.raw - rootfs Debian
# repro.c                  - reproducer C
# compilazione repro (dentro la VM Debian x86_64):
gcc -static -O0 -o repro repro.c -pthread

# avvio QEMU x86_64 in emulazione
qemu-system-x86_64 \
  -kernel ~/repro/bzImage-dc855b77 \
  -drive file=~/repro/non_bootable_disk-dc855b77.raw,format=raw,if=ide \
  -append "root=/dev/sda1 console=ttyS0 kasan.fault=panic panic=1 earlyprintk=serial" \
  -virtfs local,path=/home/luffy/repro,mount_tag=host0,security_model=mapped,id=host0 \
  -m 2G -smp 2 -nographic

# dentro la VM QEMU:
mkdir -p /mnt/host && mount -t 9p -o trans=virtio host0 /mnt/host
/mnt/host/repro
```

Il warning `lockdep` viene stampato dopo un tempo variabile (200–600 s). La riproduzione richiede un **reboot della VM tra un tentativo e l'altro** (`lockdep` disarma la classe di lock dopo il primo splat). Test statistico: **19 warning su 20 reboot** (95 %), con tempi di apparizione tra 1 s e 26 s dall'inizio del repro.

3. Analisi

`fserror_report()` (`fs/fserror.c`, introdotta 3 mesi fa da D. Wong) viene raggiunta lungo due percorsi: **sincrono** dal processo repro via `openat` quando `submit_bio_noacct` completa l'errore in linea tramite `should_fail_bio`, e **asincrono** dal loop device con completion servita da `ksoftirqd`. Solo il secondo percorso genera il warning di `lockdep`; il primo percorre la stessa call chain ma in process context.

Per entrambi i percorsi, ricostruendo la catena di chiamate dal sorgente, vale `i_count ≥ 1 ∧ I_FREEING = I_WILL_FREE = 0` all'ingresso di `fserror_report`: l'inode è tenuto vivo dalla `openat` ancora in esecuzione (sul suo stack, caso sincrone; bloccata in `folio_wait_locked_killable`, caso asincrono), e i flag `I_FREEING/I_WILL_FREE` vengono settati **unicamente** in tre punti (`iput_final`, `inode_lru_isolate`, `evict_inodes`), tutti condizionati a `i_count = 0` o a `umount` — condizioni escluse dal repro. Con `i_count ≥ 1`, `igrab()` riduce a `__iget()`, e il controllo di stato sotto `i_lock` è strutturalmente ridondante **lungo** questa call chain.

4. Fix tentato e sua limitazione

Sostituzione `igrab()` → `ihold()` in `fserror_report()` (`fs/fserror.c`, r. 158): `ihold()` incrementa `i_count` atomicamente senza prendere `i_lock`, eliminando l'IRQ-inversion. Il diff:

```
- event->inode = igrab(inode);
- if (!event->inode)
-     goto lost_event;
+ ihold(inode);
+ event->inode = inode;
```

Il ramo `lost_event` per questo caso diventa irraggiungibile: `ihold()` non ritorna nulla (è `void`) e presuppone che il chiamante tenga già una reference valida, quindi non c'è più il controllo di fallimento. Sul kernel patchato il warning di `lockdep` non si manifesta più nella stessa sequenza di riproduzione.

Il problema — discusso con il prof — è che il **contratto** di `fserror_report()` sembra più debole dell'invariante dimostrata sul repro: il commento sopra `igrab()` ("Can't iput from non-sleeping context, so grabbing another reference ...") e la presenza del ramo `lost_event` suggeriscono che l'autore ipotizzasse la possibilità di un inode già in `I_FREEING` al momento della chiamata. Se l'ipotesi è concreta per altri caller (ad es. percorsi di `iput_final` non coperti dal repro), la sostituzione con `ihold()` può far scattare il `WARN_ON(i_count == 0)` interno a `ihold()`. Per quanto si riesce a derivare dai call site attuali (`fs/iomap`, `fs/ext4`, `fs/xfs/xfs_verify_media.c`, `fs/xfs/xfs_notify_failure.c`), tutti tengono una reference attiva al momento della chiamata; ma non è possibile validarne la completezza senza il maintainer.

5. Fix upstream e stato

Il bug è stato risolto in upstream con il commit `f621324dfb3d` («iomap: fix lockdep complaint when reads fail», aprile 2026). L'approccio adottato è diverso da `igrab()` → `ihold()`: anziché modificare `fserror_report()`, il fix agisce a livello `fs/iomap/bio.c` rinominando `iomap_read_end_io` in `__iomap_read_end_io` e aggiungendo una nuova versione pubblica che, in caso di errore, accoda le bio fallite in una lista (`failed_read_list`, protetta da uno spinlock dedicato `failed_read_lock`) e le processa in un workqueue handler (`iomap_fail_reads`). In questo modo `fserror_report()` viene sempre invocata in process context, dove `igrab()` e `i_lock` sono sicuri, eliminando alla radice la possibilità di conflitto `SOFTIRQ-ON-W` / `IN-SOFTIRQ-W` indipendentemente dal caller.

Questo approccio è più robusto di `igrab()` → `ihold()` perché non si affida all'invariante `i_count ≥ 1` al momento della chiamata: sposta il problema fuori dal contesto di interruzione piuttosto che modificare il contratto di locking.

Il fix è ora in upstream/master.